

Multi-Portfolio Architecture Extension

Scaling from Single to Multiple Portfolios

February 2026 | Complete Multi-Portfolio Framework

TABLE OF CONTENTS

1. [Overview](#)
 2. [What Stays the Same](#)
 3. [What Needs to Change](#)
 4. [Portfolio Configuration](#)
 5. [Data Architecture Changes](#)
 6. [Flexible Universe Screening](#)
 7. [Script Modifications Required](#)
 8. [Aggregate Monitoring](#)
 9. [Reporting Structure](#)
 10. [Operational Workflow](#)
 11. [Implementation Roadmap](#)
-

OVERVIEW

Current State: Single Portfolio

Current Architecture:

- One portfolio with one set of parameters
- One rebalancing process
- One set of positions
- One performance report

Limitations:

- Cannot test different strategies simultaneously
 - Cannot segregate capital by risk profile
 - Cannot compare strategy variants
 - All capital allocated to one approach
-

Future State: Multiple Portfolios

Target Architecture:

- N portfolios (currently 2, scalable to 10+)
- Each portfolio can have:
 - Different parameters (SMA periods, position count, etc.)
 - Different risk limits (max drawdown, position size)
 - Different universe filters (stocks only vs stocks+ETFs)
 - Different capital allocation
 - Different rebalancing frequency (monthly, quarterly)
- Aggregate monitoring across all portfolios
- Consolidated and individual reporting

Benefits:

- **Diversification:** Different strategies reduce correlation
 - **Testing:** Run parameter variants in parallel (production A/B testing)
 - **Risk Segmentation:** Conservative portfolio + aggressive portfolio
 - **Client Separation:** Manage multiple client accounts
 - **Strategy Evolution:** Test new approaches without disrupting main portfolio
-

WHAT STAYS THE SAME

Shared Infrastructure

These components remain unchanged and serve all portfolios:

1. Data Layer (Scripts 1-5)

No Changes Required

- Data download is shared (one download serves all portfolios)
- Universe screening can be portfolio-specific but uses same data
- Indicators calculated once, used by all portfolios
- Fundamentals data shared

Rationale: No point downloading same data multiple times

2. Backtesting & Validation (Scripts 16-21)

No Changes Required

- Each portfolio configuration backtested independently
- Same validation framework applied to each
- Each portfolio gets its own deployment decision

Rationale: Validation framework is universal

3. Visualization (Script 15)

Minor Enhancement

- Add portfolio filter to chart generation
- Otherwise unchanged

WHAT NEEDS TO CHANGE

Portfolio-Specific Components

1. Portfolio Configuration Files

New: Portfolio configuration system

2. Strategy Parameters

Change: Parameters now per-portfolio, not global

3. Position Tracking

Change: Positions tagged with portfolio_id

4. Rebalancing Process

Change: Run once per portfolio (or in batch)

5. Risk Monitoring

Change: Monitor each portfolio + aggregate

6. Reporting

Change: Individual reports + consolidated views

7. Execution Logging

Change: Trades tagged with portfolio_id

8. Performance Management

Change: Track performance per portfolio + aggregate

PORTFOLIO CONFIGURATION

Portfolio Definition Schema

File: config/portfolios.json

```
{
  "portfolios": {
    "CONSERVATIVE": {
      "portfolio_id": "CONSERVATIVE",
      "display_name": "Conservative Growth Portfolio",
      "description": "Low volatility, dividend-focused strategy",

```

```

"active": true,
"initial_capital": 50000,
"currency": "EUR",

"strategy_parameters": {
  "sma_fast": 50,
  "sma_slow": 200,
  "adx_threshold": 20,
  "initial_stop_multiplier": 3.0,
  "trailing_stop_multiplier": 4.0,
  "trailing_stop_activation": 0.15,
  "position_count_min": 15,
  "position_count_max": 25,
  "risk_per_position": 0.015,
  "rebalancing_frequency": "monthly"
},

"universe_filters": {
  "asset_classes": ["stocks", "etfs"],
  "exchanges": ["NYSE", "NASDAQ", "XETRA", "LSE"],
  "min_market_cap": 1000000000,
  "min_avg_volume": 5000000,
  "min_price": 10.00,
  "exclude_sectors": [],
  "include_sectors": [],
  "max_beta": 1.2,
  "min_dividend_yield": 0.02
},

"risk_limits": {
  "max_position_size": 0.06,
  "min_position_size": 0.005,
  "max_sector_concentration": 0.25,
  "max_asset_class_stocks": 0.60,
  "max_asset_class_etfs": 0.50,
  "max_asset_class_crypto": 0.00,
  "max_drawdown_halt": -0.20,
  "max_portfolio_beta": 1.0,
  "min_cash_reserve": 0.05
},

"circuit_breakers": {
  "portfolio_drawdown": -0.12,
  "vix_spike": 40,
  "correlation_breakdown": 0.85,
  "concentration_creep": 0.25,
  "data_staleness_days": 3
},

"rebalancing_schedule": {
  "frequency": "monthly",
  "anchor": "last_trading_day",
  "execution_day": "first_trading_day_next_month"
},

"reporting": {
  "benchmark": "SPY",
  "performance_target_annual": 0.10,
  "max_drawdown_target": -0.15,
  "sharpe_target": 1.0
}
},

"AGGRESSIVE": {
  "portfolio_id": "AGGRESSIVE",
  "display_name": "Aggressive Growth Portfolio",
  "description": "High growth, momentum-focused strategy",
  "active": true,
  "initial_capital": 50000,
  "currency": "EUR",

  "strategy_parameters": {
    "sma_fast": 30,
    "sma_slow": 150,
    "adx_threshold": 25,
    "initial_stop_multiplier": 2.5,
    "trailing_stop_multiplier": 3.5,
    "trailing_stop_activation": 0.20,
    "position_count_min": 10,
    "position_count_max": 15,
    "risk_per_position": 0.025,
    "rebalancing_frequency": "monthly"
  },

```

```
"universe_filters": {
  "asset_classes": ["stocks", "etfs", "crypto"],
  "exchanges": ["NYSE", "NASDAQ"],
  "min_market_cap": 500000000,
  "min_avg_volume": 10000000,
  "min_price": 5.00,
  "exclude_sectors": ["Utilities", "Consumer Staples"],
  "include_sectors": [],
  "max_beta": null,
  "min_dividend_yield": null
},

"risk_limits": {
  "max_position_size": 0.10,
  "min_position_size": 0.005,
  "max_sector_concentration": 0.35,
  "max_asset_class_stocks": 0.70,
  "max_asset_class_etfs": 0.50,
  "max_asset_class_crypto": 0.20,
  "max_drawdown_halt": -0.30,
  "max_portfolio_beta": 1.5,
  "min_cash_reserve": 0.02
},

"circuit_breakers": {
  "portfolio_drawdown": -0.20,
  "vix_spike": 50,
  "correlation_breakdown": 0.90,
  "concentration_creep": 0.35,
  "data_staleness_days": 3
},

"rebalancing_schedule": {
  "frequency": "monthly",
  "anchor": "last_trading_day",
  "execution_day": "first_trading_day_next_month"
},

"reporting": {
  "benchmark": "QQQ",
  "performance_target_annual": 0.18,
  "max_drawdown_target": -0.25,
  "sharpe_target": 1.2
}
}
},

"global_settings": {
  "transaction_cost_pct": 0.001,
  "slippage_stocks_pct": 0.0005,
  "slippage_etfs_pct": 0.001,
  "slippage_crypto_pct": 0.002,
  "api_keys_file": ".env",
  "timezone": "Europe/Amsterdam"
}
}
```

Portfolio Parameter Comparison

Parameter	Conservative	Aggressive	Rationale
SMA Fast	50	30	Aggressive responds faster to trends
SMA Slow	200	150	Aggressive more sensitive
ADX Threshold	20	25	Aggressive requires stronger trends
Initial Stop	3.0× ATR	2.5× ATR	Aggressive tighter stops
Trailing Stop	4.0× ATR	3.5× ATR	Aggressive protects profits sooner
Position Count	15-25	10-15	Conservative more diversified
Risk Per Position	1.5%	2.5%	Aggressive larger bets
Max Position Size	6%	10%	Aggressive allows concentration
Max Drawdown Halt	-20%	-30%	Conservative stops earlier
Asset Classes	Stocks, ETFs	Stocks, ETFs, Crypto	Aggressive includes crypto
Benchmark	SPY	QQQ	Different risk profiles

DATA ARCHITECTURE CHANGES

Modified Directory Structure

```
trend_strategy/
├── config/
│   ├── exchanges.json (unchanged)
│   ├── filter_thresholds.json (deprecated - moved to portfolios.json)
│   ├── portfolios.json ★ NEW
│   └── strategy_parameters.json (deprecated - moved to portfolios.json)
├── data_cache/ (shared across all portfolios)
│   ├── raw_bulk/ (unchanged)
│   ├── fundamentals/ (unchanged)
│   ├── consolidated/ (unchanged)
│   ├── indicators/ (unchanged)
│   ├── qualified/ ★ MODIFIED
│   │   ├── CONSERVATIVE_qualified_symbols.json
│   │   ├── AGGRESSIVE_qualified_symbols.json
│   │   └── ALL_qualified_symbols.json (union of all)
│   ├── signals/ ★ MODIFIED
│   │   ├── CONSERVATIVE_momentum_ranked.json
│   │   ├── AGGRESSIVE_momentum_ranked.json
│   │   └── timestamp.json
│   └── portfolio/ ★ MODIFIED
│       ├── CONSERVATIVE_position_sizes.json
│       └── AGGRESSIVE_position_sizes.json
├── data/ ★ RESTRUCTURED
│   ├── portfolios/
│   │   ├── CONSERVATIVE/
│   │   │   ├── portfolio_state.json
│   │   │   ├── positions/
│   │   │   │   └── {YYYY-MM-DD}_positions.json
│   │   │   └── executions/
│   │   │       └── {YYYY-MM-DD}_trades.json
│   │   ├── AGGRESSIVE/
│   │   │   ├── portfolio_state.json
│   │   │   ├── positions/
│   │   │   │   └── {YYYY-MM-DD}_positions.json
│   │   │   └── executions/
│   │   │       └── {YYYY-MM-DD}_trades.json
│   │   └── AGGREGATE/
│   │       ├── aggregate_state.json
│   │       └── correlation_matrix.json
│   └── performance/
│       ├── CONSERVATIVE/
│       │   ├── attribution/
│       │   ├── risk/
│       │   └── dashboard/
│       ├── AGGRESSIVE/
│       │   ├── attribution/
│       │   ├── risk/
│       │   └── dashboard/
│       └── AGGREGATE/
│           ├── consolidated_performance.json
│           └── cross_portfolio_analysis.json
├── reports/ ★ RESTRUCTURED
│   ├── rebalancing/
│   │   ├── CONSERVATIVE/
│   │   │   └── {YYYY-MM}_recommendations.pdf
│   │   ├── AGGRESSIVE/
│   │   │   └── {YYYY-MM}_recommendations.pdf
│   │   └── CONSOLIDATED/
│   │       └── {YYYY-MM}_all_portfolios.pdf
│   ├── daily/
│   │   ├── CONSERVATIVE/
│   │   ├── AGGRESSIVE/
│   │   └── AGGREGATE/
│   └── performance/
│       ├── CONSERVATIVE/
│       ├── AGGRESSIVE/
│       └── CONSOLIDATED/
└── logs/ ★ MODIFIED
    ├── CONSERVATIVE/
    ├── AGGRESSIVE/
    └── SYSTEM/
```

Portfolio State Schema

File: data/portfolios/{PORTFOLIO_ID}/portfolio_state.json

```
{
  "portfolio_id": "CONSERVATIVE",
  "as_of_date": "2026-02-21",
  "cash": 15234.50,
  "equity_value": 62450.75,
  "total_value": 77685.25,
  "initial_capital": 50000.00,
  "total_return": 0.5537,
  "positions": [
    {
      "symbol": "AAPL.US",
      "entry_date": "2026-01-03",
      "entry_price": 182.50,
      "current_price": 195.20,
      "shares": 25,
      "position_value": 4880.00,
      "cost_basis": 4562.50,
      "unrealized_pnl": 317.50,
      "unrealized_pnl_pct": 0.0696,
      "initial_stop": 175.00,
      "current_stop": 185.50,
      "trailing_stop_active": true,
      "portfolio_weight": 0.0628
    }
  ],
  "metadata": {
    "last_rebalancing_date": "2026-01-31",
    "next_rebalancing_date": "2026-02-28",
    "positions_count": 18,
    "circuit_breakers_active": false,
    "last_updated": "2026-02-21T18:00:00Z"
  }
}
```

Aggregate State Schema

File: data/portfolios/AGGREGATE/aggregate_state.json

```
{
  "as_of_date": "2026-02-21",
  "portfolios": {
    "CONSERVATIVE": {
      "total_value": 77685.25,
      "cash": 15234.50,
      "equity": 62450.75,
      "return_ytd": 0.1254,
      "return_inception": 0.5537,
      "sharpe_ratio": 1.15,
      "max_drawdown": -0.08,
      "positions_count": 18,
      "beta": 0.85
    },
    "AGGRESSIVE": {
      "total_value": 68420.50,
      "cash": 3210.30,
      "equity": 65210.20,
      "return_ytd": 0.1845,
      "return_inception": 0.3684,
      "sharpe_ratio": 1.32,
      "max_drawdown": -0.15,
      "positions_count": 12,
      "beta": 1.25
    }
  },
  "aggregate": {
    "total_value": 146105.75,
    "total_cash": 18444.80,
    "total_equity": 127660.95,
    "weighted_return_ytd": 0.1528,
    "weighted_sharpe": 1.23,
    "combined_max_drawdown": -0.11,
    "cross_portfolio_correlation": 0.62,
    "total_positions": 30,
    "overlapping_positions": 5
  },
}
```

```
"cross_portfolio_analysis": {
  "diversification_benefit": 0.15,
  "correlation_matrix": {
    "CONSERVATIVE_AGGRESSIVE": 0.62
  },
  "overlapping_symbols": [
    "AAPL.US",
    "MSFT.US",
    "GOOGL.US",
    "NVDA.US",
    "META.US"
  ]
}
```

FLEXIBLE UNIVERSE SCREENING

Overview

Why Portfolio-Specific Universe Screening?

Each portfolio can now target different market segments with its own screening criteria:

- **CONSERVATIVE:** Large-cap, high-liquidity, dividend-paying, defensive sectors
- **AGGRESSIVE:** Mid/small-cap, high-growth, momentum stocks, tech-heavy
- **INCOME:** High dividend yield, dividend aristocrats, stable cash flow
- **GROWTH:** Tech/innovation sectors, high revenue growth, no dividend requirement

Benefits:

1. **True Diversification:** Different opportunity sets reduce correlation
2. **Specialized Strategies:** Each portfolio optimized for its objective
3. **Risk Segmentation:** Match universe to risk profile
4. **Opportunity Coverage:** Capture different market segments simultaneously

Universe Criteria Categories

1. Exchange & Geographic Filters

Available Exchanges:

- NYSE, NASDAQ (US)
- XETRA (Germany)
- LSE (UK)
- PA (Euronext Paris)
- AS (Euronext Amsterdam)

Configuration:

```
"universe_filters": {
  "exchanges": ["NYSE", "NASDAQ"],
  "countries": ["US"],
  "exclude_regions": []
}
```

Use Cases:

- US-Only Portfolio: NYSE, NASDAQ only
- European Portfolio: XETRA, LSE, PA, AS only
- Global Portfolio: All exchanges

2. Market Capitalization Filters

Market Cap Tiers:

- **Mega-cap:** > €200 billion
- **Large-cap:** €10B - €200B
- **Mid-cap:** €2B - €10B
- **Small-cap:** €500M - €2B
- **Micro-cap:** €50M - €500M

Configuration:

```
"universe_filters": {
  "min_market_cap": 1000000000,    // €10B (large-cap minimum)
  "max_market_cap": null,          // No maximum
  "market_cap_tier": "large-cap"    // Or: "mid-cap", "small-cap", "all"
}
```

Risk Characteristics:

- Mega/Large-cap: Lower volatility, higher liquidity, lower growth
- Mid-cap: Moderate volatility, good liquidity, balanced growth
- Small-cap: Higher volatility, lower liquidity, higher growth potential

3. Liquidity Filters

Configuration:

```
"universe_filters": {
  "min_avg_volume_30d": 1000000,    // 1M shares/day
  "min_avg_dollar_volume_30d": 1000000, // €10M/day
  "max_bid_ask_spread_pct": 0.005    // 0.5% maximum spread
}
```

Portfolio-Specific Examples:

- Large Portfolio (€5M+): min_dollar_volume €50M/day
- Small Portfolio (€50K): min_dollar_volume €5M/day
- Aggressive: min_dollar_volume €2M/day (willing to trade less liquid)

4. Price Filters

Configuration:

```
"universe_filters": {
  "min_price": 10.00,              // €10 minimum (quality filter)
  "max_price": null,              // No maximum
  "exclude_penny_stocks": true    // Exclude if <€5 ever in last year
}
```

Portfolio-Specific:

- Conservative: min_price €15 (avoid volatility of low-price stocks)
- Aggressive: min_price €5 (willing to trade cheaper stocks)

5. Dividend Filters

Configuration:

```
"universe_filters": {
  "min_dividend_yield": 0.02,      // 2% minimum yield
  "dividend_consistency": 5,       // Paid dividends for 5+ years
  "dividend_growth_required": true // Must have growing dividend
}
```

Use Cases:

- Income portfolios: Require high yield + consistency
- Growth portfolios: Set to null (ignore dividends)
- Dividend Growth: Require growing dividends (aristocrats)

6. Profitability Filters

Configuration:

```
"universe_filters": {
  "require_positive_earnings": true, // Must be profitable
  "min_roe": 0.10,                  // 10% minimum ROE
  "min_profit_margin": 0.05,        // 5% minimum profit margin
}
```

```
"max_debt_to_equity": 2.0          // Leverage limit
}
```

Portfolio-Specific:

- Conservative: Require profitability, high ROE, low debt
 - Growth: May allow unprofitable (fast-growing companies)
-

7. Growth Filters

Configuration:

```
"universe_filters": {
  "min_revenue_growth_3y": 0.15,    // 15%+ revenue growth
  "min_earnings_growth_3y": 0.20,   // 20%+ earnings growth
  "min_eps_growth_yoy": 0.10        // 10%+ EPS growth year-over-year
}
```

Use Cases:

- Growth portfolios: High revenue/earnings growth required
 - Value portfolios: Lower or no growth requirement
 - Balanced: Moderate growth requirement
-

8. Sector & Industry Filters

GICS Sectors:

1. Energy
2. Materials
3. Industrials
4. Consumer Discretionary
5. Consumer Staples
6. Health Care
7. Financials
8. Information Technology
9. Communication Services
10. Utilities
11. Real Estate

Configuration:

```
"universe_filters": {
  "include_sectors": ["Information Technology", "Communication Services"],
  "exclude_sectors": ["Utilities", "Real Estate"],
  "min_sector_diversification": 3,    // Must have ≥3 sectors represented
  "max_sector_concentration": 0.40   // No sector >40% of universe
}
```

Use Cases:

- Sector-focused portfolio: Include only specific sectors
 - Defensive portfolio: Exclude cyclicals (Energy, Materials)
 - Diversified portfolio: Include all, enforce diversification
-

9. Beta & Risk Filters

Configuration:

```
"universe_filters": {
  "min_beta": 0.5,                // Not too defensive
  "max_beta": 1.5,                // Not too aggressive
  "max_volatility_252d": 0.40,    // Annual volatility <40%
  "min_sharpe_1y": 0.5            // Historical Sharpe >0.5
}
```

Beta Interpretation:

- Beta = 1.0: Moves with market
- Beta > 1.0: More volatile than market

- Beta < 1.0: Less volatile than market

Portfolio-Specific:

- Conservative: max_beta 1.0 (defensive)
- Aggressive: min_beta 1.0, max_beta 2.5 (volatile ok)

10. Quality & Listing Filters

Configuration:

```
"universe_filters": {
  "min_listing_days": 252,           // 1 year minimum history
  "exclude_spacs": true,             // No SPACs
  "exclude_otc": true,               // No OTC stocks
  "require_audited_financials": true, // Must have audited statements
  "exclude_chinese_vies": true,      // No Variable Interest Entities
  "max_insider_ownership": 0.75      // No >75% insider control
}
```

Portfolio-Specific:

- Conservative: min_listing_days 1260 (5 years, established companies)
- Aggressive: min_listing_days 126 (6 months, allow IPOs)

Complete Portfolio Configuration Examples

Example 1: CONSERVATIVE (Dividend Income)

```
{
  "portfolio_id": "CONSERVATIVE",
  "display_name": "Conservative Dividend Income",
  "objective": "Generate stable income with capital preservation",

  "universe_filters": {
    // Geographic
    "exchanges": ["NYSE", "NASDAQ", "XETRA", "LSE"],
    "countries": ["US", "DE", "GB"],

    // Size & Liquidity
    "min_market_cap": 1000000000,      // €10B (large-cap only)
    "max_market_cap": null,
    "min_avg_dollar_volume_30d": 5000000, // €50M/day (very liquid)
    "max_bid_ask_spread_pct": 0.003,    // 0.3% max spread

    // Price
    "min_price": 15.00,                 // €15 minimum

    // Dividends (KEY CRITERIA)
    "min_dividend_yield": 0.025,        // 2.5% minimum yield
    "dividend_consistency": 10,          // 10 years of dividends
    "dividend_growth_required": true,    // Growing dividends

    // Profitability
    "require_positive_earnings": true,
    "min_roe": 0.12,                    // 12% ROE minimum
    "max_debt_to_equity": 1.5,           // Conservative leverage

    // Sectors (Defensive)
    "include_sectors": [],               // All sectors allowed
    "exclude_sectors": ["Energy"],       // Exclude cyclical energy

    // Asset Classes
    "asset_classes": ["stocks", "etfs"], // No crypto
    "exclude_leveraged_etfs": true,

    // Risk
    "max_beta": 1.0,                     // Beta ≤1.0 (defensive)
    "max_volatility_252d": 0.25,         // Max 25% annual vol

    // Quality
    "min_listing_days": 1260,            // 5 years minimum (established)
    "exclude_spacs": true
  }
}
```

Resulting Universe:

- ~200-300 large-cap dividend aristocrats
- Low volatility, high quality
- Defensive sectors weighted
- Examples: JNJ, PG, KO, PEP, WMT

Example 2: AGGRESSIVE (Growth Momentum)

```
{
  "portfolio_id": "AGGRESSIVE",
  "display_name": "Aggressive Growth Momentum",
  "objective": "Maximize capital appreciation through high-growth stocks",

  "universe_filters": {
    // Geographic
    "exchanges": ["NYSE", "NASDAQ"],      // US-only for growth
    "countries": ["US"],

    // Size & Liquidity
    "min_market_cap": 500000000,          // €500M (include small-cap)
    "max_market_cap": 50000000000,        // €50B max (exclude mega-cap)
    "min_avg_dollar_volume_30d": 10000000, // €10M/day (moderate liquidity)
    "max_bid_ask_spread_pct": 0.01,        // 1% max spread

    // Price
    "min_price": 5.00,                    // €5 minimum (allow cheaper stocks)

    // Dividends (NOT REQUIRED)
    "min_dividend_yield": null,            // No dividend requirement
    "dividend_consistency": null,

    // Profitability (GROWTH FOCUS)
    "require_positive_earnings": false,    // Growth stocks may not be profitable yet
    "min_revenue_growth_3y": 0.25,         // 25%+ revenue growth (KEY)

    // Sectors (Growth-oriented)
    "include_sectors": [
      "Information Technology",
      "Communication Services",
      "Health Care",                      // Biotech
      "Consumer Discretionary"            // E-commerce, innovation
    ],
    "exclude_sectors": [
      "Utilities",
      "Consumer Staples",
      "Real Estate"
    ],

    // Asset Classes
    "asset_classes": ["stocks", "crypto"], // Include crypto!
    "max_crypto_percentage": 0.20,         // Up to 20% crypto

    // Risk (AGGRESSIVE)
    "min_beta": 1.0,                      // Beta ≥1.0 (volatile ok)
    "max_beta": 2.5,                      // Up to 2.5× market vol
    "max_volatility_252d": 0.80,          // Up to 80% annual vol

    // Quality
    "min_listing_days": 126,              // 6 months (allow IPOs)
    "exclude_spacs": false                // SPACs allowed
  }
}
```

Resulting Universe:

- ~400-600 growth stocks (small to mid-cap)
- High volatility, high growth potential
- Tech/innovation heavy
- Examples: PLTR, SNOW, CRWD, ZS, DDOG, RBLX

Example 3: BALANCED (Core Holdings)

```
{
  "portfolio_id": "BALANCED",
  "display_name": "Balanced Core Portfolio",
```

```

"objective": "Balanced growth and income with moderate risk",

"universe_filters": {
  // Geographic
  "exchanges": ["NYSE", "NASDAQ", "XETRA", "LSE", "PA", "AS"],
  "countries": null, // All countries

  // Size & Liquidity
  "min_market_cap": 2000000000, // €2B (mid-cap minimum)
  "max_market_cap": null,
  "min_avg_dollar_volume_30d": 20000000, // €20M/day

  // Price
  "min_price": 8.00,

  // Dividends (MODERATE)
  "min_dividend_yield": 0.01, // 1%+ yield (some income)

  // Profitability (BALANCED)
  "require_positive_earnings": true, // Must be profitable
  "min_roe": 0.08, // 8% ROE minimum
  "min_revenue_growth_3y": 0.05, // 5%+ growth (modest)

  // Sectors (DIVERSIFIED)
  "include_sectors": [], // All sectors
  "exclude_sectors": [], // No exclusions
  "min_sector_diversification": 5, // Must have ≥5 sectors

  // Asset Classes
  "asset_classes": ["stocks", "etfs"],
  "max_etf_percentage": 0.30,

  // Risk (MODERATE)
  "min_beta": 0.7,
  "max_beta": 1.3,
  "max_volatility_252d": 0.35, // 35% max annual vol

  // Quality
  "min_listing_days": 504, // 2 years minimum
  "exclude_spacs": true
}
}

```

Resulting Universe:

- ~800-1200 quality mid-to-large cap stocks
- Diversified across sectors
- Balanced growth/income mix
- Examples: MSFT, AAPL, GOOGL, V, MA, UNH

Universe Overlap Strategy

Question: Should portfolio universes overlap or be mutually exclusive?

RECOMMENDED: Overlapping with Monitoring

Approach:

- All portfolios screen from same base universe
- Different filters lead to different qualified sets
- Some symbols may qualify for multiple portfolios (natural)

Example:

Base Universe: 3,000 symbols

CONSERVATIVE filters → 300 qualified

AGGRESSIVE filters → 450 qualified

Overlap: 80 symbols (qualify for both)

CONSERVATIVE can hold: Any of its 300

AGGRESSIVE can hold: Any of its 450

Both can hold AAPL if it qualifies for both

Pros:

- ✓ Natural - best opportunities may suit multiple strategies
- ✓ Flexible - can adjust allocation per portfolio
- ✓ Realistic - allows position in both if momentum strong

Cons:

- ✗ Potential correlation if too much overlap
- ✗ Need to monitor cross-portfolio exposure

Mitigation:

```
"aggregate_risk_limits": {  
  "max_cross_portfolio_overlap_pct": 0.30, // Alert if >30% overlap  
  "max_single_symbol_total_exposure": 0.08, // Max 8% total across all portfolios  
  "alert_if_overlap_high_correlation": true // Alert if overlapping positions correlate >0.8  
}
```

Example Scenario:

CONSERVATIVE holds AAPL: 3% of portfolio (€2,310)
AGGRESSIVE holds AAPL: 5% of portfolio (€3,420)

Total AAPL across both: €5,730
Total capital: €100,000
Total exposure to AAPL: 5.73%

✓ PASS: Below 8% limit

Alternative: Non-Overlapping Universes

Approach:

- Define mutually exclusive universe segments
- Each portfolio gets unique opportunity set

Example:

CONSERVATIVE: Large-cap only (>€10B)
AGGRESSIVE: Mid-cap only (€2B-€10B)
SMALL_CAP: Small-cap only (€500M-€2B)

No overlap possible

Pros:

- ✓ Perfect diversification across market segments
- ✓ No cross-portfolio position risk

Cons:

- ✗ Artificial - may miss best opportunities
- ✗ Inflexible - can't hold AAPL in multiple portfolios

Modified Script 4: Universe Screener

Current (Single Universe)

```
screen_universe(as_of_date) →  
  Load global filter_thresholds.json →  
  Apply filters to all symbols →  
  Save qualified_symbols.json
```

Modified (Multi-Portfolio)

```
screen_universe(portfolio_id, as_of_date) →  
  Load portfolio config (portfolios.json) →  
  Extract universe_filters for this portfolio →  
  Apply portfolio-specific filters in sequence →  
  Save {PORTFOLIO_ID}_qualified_symbols.json
```

Filter Application Sequence

Order matters for performance (most restrictive first):

1. **Exchange Filter** (quick, reduces set significantly)
2. **Asset Class Filter** (stocks vs ETFs vs crypto)
3. **Market Cap Filter** (numeric comparison)
4. **Liquidity Filters** (requires price/volume data)
5. **Price Filter** (quick numeric check)
6. **Sector Filters** (requires fundamental data)
7. **Dividend Filters** (requires historical dividend data)

- 8. **Profitability Filters** (requires financial statements)
- 9. **Growth Filters** (requires 3-year historical financials)
- 10. **Beta & Risk Filters** (requires return history)
- 11. **Quality Filters** (listing days, SPAC status, etc.)

Example Implementation Flow

Initial symbols: 3,000
After exchange filter: 2,100 (NYSE, NASDAQ only)
After asset class filter: 1,950 (stocks only, no ETFs/crypto)
After market cap filter: 650 (>€10B only)
After liquidity filter: 580 (>€50M daily volume)
After price filter: 570 (>€15)
After sector filter: 420 (exclude Energy)
After dividend filter: 320 (>2.5% yield, 10yr consistency)
After profitability filter: 305 (ROE >12%, D/E <1.5)
After beta filter: 287 (beta <1.0)
After quality filter: 287 (all pass 5yr listing requirement)
Final qualified: 287 symbols for CONSERVATIVE portfolio

Screening Report Generation

Auto-generate screening summary per portfolio:

File: reports/screening/{PORTFOLIO_ID}_{DATE}_screening_report.json

```
{
  "portfolio_id": "CONSERVATIVE",
  "as_of_date": "2026-02-21",
  "filters_applied": {
    "exchanges": ["NYSE", "NASDAQ", "XETRA", "LSE"],
    "min_market_cap": 1000000000,
    "min_dividend_yield": 0.025,
    // ... all filters
  },
  "qualified_count": 287,
  "filter_impact": {
    "initial_symbols": 3000,
    "after_exchange_filter": 2100,
    "after_asset_class_filter": 1950,
    "after_market_cap_filter": 650,
    "after_liquidity_filter": 580,
    "after_price_filter": 570,
    "after_sector_filter": 420,
    "after_dividend_filter": 320,
    "after_profitability_filter": 305,
    "after_beta_filter": 287,
    "final_qualified": 287
  },
  "breakdown": {
    "by_exchange": {
      "NYSE": 158,
      "NASDAQ": 95,
      "XETRA": 22,
      "LSE": 12
    },
    "by_sector": {
      "Information Technology": 62,
      "Health Care": 48,
      "Financials": 41,
      "Consumer Staples": 35,
      "Industrials": 32,
      "Communication Services": 28,
      "Consumer Discretionary": 25,
      "Materials": 10,
      "Real Estate": 6
    },
    "by_market_cap_tier": {
      "mega_cap": 42,
      "large_cap": 245,
      "mid_cap": 0,
      "small_cap": 0
    },
    "by_dividend_yield": {
      "2-3%": 125,
      "3-4%": 98,
      "4-5%": 47,
```

```
    ">5%": 17
  }
}
```

Cross-Portfolio Universe Analysis

Track overlap across all portfolios:

File: reports/screening/AGGREGATE_{DATE}_universe_overlap.json

```
{
  "as_of_date": "2026-02-21",
  "portfolios": ["CONSERVATIVE", "AGGRESSIVE"],
  "universe_sizes": {
    "CONSERVATIVE": 287,
    "AGGRESSIVE": 412
  },
  "overlaps": {
    "CONSERVATIVE_AGGRESSIVE": {
      "symbols": ["AAPL.US", "MSFT.US", "GOOGL.US", ...],
      "count": 78,
      "percentage": 0.126, // 12.6% overlap
      "universe_1_size": 287,
      "universe_2_size": 412
    }
  },
  "unique_to_portfolio": {
    "CONSERVATIVE": 209, // Symbols only CONSERVATIVE can trade
    "AGGRESSIVE": 334 // Symbols only AGGRESSIVE can trade
  },
  "analysis": {
    "diversification_benefit": "GOOD",
    "overlap_percentage": 0.126,
    "recommendation": "Overlap is reasonable (<30%). Portfolios targeting different market segments."
  }
}
```

SCRIPT MODIFICATIONS REQUIRED

Scripts That Need Modification

Scripts 1-3: Data Download & Consolidation

Status: No changes required ✓

Data download and consolidation is shared - same download process serves all portfolios.

Script 4: Universe Screener

Status: Major modification required ⚠️

Current:

```
def screen_universe(as_of_date: str):
    # Uses global filter_thresholds.json
    thresholds = load_filter_thresholds()

    # Apply same filters to all symbols
    qualified = apply_filters(all_symbols, thresholds)

    # Save single qualified list
    save_qualified_symbols(qualified, as_of_date)
```

Modified:

```
def screen_universe(portfolio_id: str, as_of_date: str):
    # Load portfolio-specific configuration
    config = load_portfolio_config(portfolio_id)
    filters = config['universe_filters']

    logger.info(f"Screening universe for portfolio: {portfolio_id}")

    # Get all symbols
```

```

all_symbols = get_all_symbols_from_cache(as_of_date)
qualified = all_symbols

# Apply filters in sequence (most restrictive first)

# 1. Exchange Filter
if filters.get('exchanges'):
    qualified = [s for s in qualified if get_exchange(s) in filters['exchanges']]
    logger.info(f"After exchange filter: {len(qualified)}")

# 2. Asset Class Filter
if filters.get('asset_classes'):
    qualified = [s for s in qualified if get_asset_class(s) in filters['asset_classes']]
    logger.info(f"After asset class filter: {len(qualified)}")

# 3. Market Cap Filter
if filters.get('min_market_cap'):
    qualified = apply_market_cap_filter(
        qualified,
        min_cap=filters['min_market_cap'],
        max_cap=filters.get('max_market_cap'),
        as_of_date=as_of_date
    )
    logger.info(f"After market cap filter: {len(qualified)}")

# 4. Liquidity Filter
if filters.get('min_avg_dollar_volume_30d'):
    qualified = apply_liquidity_filter(
        qualified,
        min_dollar_volume=filters['min_avg_dollar_volume_30d'],
        max_spread=filters.get('max_bid_ask_spread_pct'),
        as_of_date=as_of_date
    )
    logger.info(f"After liquidity filter: {len(qualified)}")

# 5. Price Filter
if filters.get('min_price'):
    qualified = apply_price_filter(
        qualified,
        min_price=filters['min_price'],
        max_price=filters.get('max_price'),
        as_of_date=as_of_date
    )
    logger.info(f"After price filter: {len(qualified)}")

# 6. Sector Filter
if filters.get('include_sectors') or filters.get('exclude_sectors'):
    qualified = apply_sector_filter(
        qualified,
        include_sectors=filters.get('include_sectors', []),
        exclude_sectors=filters.get('exclude_sectors', []),
        as_of_date=as_of_date
    )
    logger.info(f"After sector filter: {len(qualified)}")

# 7. Dividend Filter
if filters.get('min_dividend_yield'):
    qualified = apply_dividend_filter(
        qualified,
        min_yield=filters['min_dividend_yield'],
        consistency_years=filters.get('dividend_consistency'),
        growth_required=filters.get('dividend_growth_required', False),
        as_of_date=as_of_date
    )
    logger.info(f"After dividend filter: {len(qualified)}")

# 8. Profitability Filter
if filters.get('require_positive_earnings'):
    qualified = apply_profitability_filter(
        qualified,
        require_positive=True,
        min_roe=filters.get('min_roe'),
        min_profit_margin=filters.get('min_profit_margin'),
        as_of_date=as_of_date
    )
    logger.info(f"After profitability filter: {len(qualified)}")

# 9. Growth Filter
if filters.get('min_revenue_growth_3y') or filters.get('min_earnings_growth_3y'):
    qualified = apply_growth_filter(
        qualified,
        min_revenue_growth=filters.get('min_revenue_growth_3y'),
        min_earnings_growth=filters.get('min_earnings_growth_3y'),

```

```

        as_of_date=as_of_date
    )
    logger.info(f"After growth filter: {len(qualified)}")

# 10. Beta & Risk Filter
if filters.get('max_beta') or filters.get('min_beta'):
    qualified = apply_beta_filter(
        qualified,
        min_beta=filters.get('min_beta'),
        max_beta=filters.get('max_beta'),
        max_volatility=filters.get('max_volatility_252d'),
        as_of_date=as_of_date
    )
    logger.info(f"After beta/risk filter: {len(qualified)}")

# 11. Quality Filter
if filters.get('min_listing_days'):
    qualified = apply_quality_filter(
        qualified,
        min_listing_days=filters['min_listing_days'],
        exclude_spacs=filters.get('exclude_spacs', True),
        as_of_date=as_of_date
    )
    logger.info(f"After quality filter: {len(qualified)}")

logger.info(f"Final qualified symbols for {portfolio_id}: {len(qualified)}")

# Save with portfolio ID
save_qualified_symbols(qualified, portfolio_id, as_of_date)

# Generate screening report
generate_screening_report(portfolio_id, qualified, filters, as_of_date)

return qualified

```

Key Changes:

- Accepts portfolio_id parameter
- Loads portfolio-specific filters from portfolios.json
- Applies 11 categories of filters in sequence
- Saves results to {PORTFOLIO_ID}_qualified_symbols.json
- Generates screening report showing filter impact
- Each portfolio gets its own qualified universe

New Filter Functions Required:

- apply_market_cap_filter()
- apply_liquidity_filter()
- apply_price_filter()
- apply_sector_filter()
- apply_dividend_filter()
- apply_profitability_filter()
- apply_growth_filter()
- apply_beta_filter()
- apply_quality_filter()

Script 5: Indicator Calculator

Status: No changes required ✓

Indicators calculated once, used by all portfolios. No portfolio-specific logic needed.

Script 6: Trend Qualifier

Status: Minor modification required

Current:

```

def qualify_trends(as_of_date: str):
    # Uses global filter_thresholds.json
    thresholds = load_filter_thresholds()
    # ... rest of logic

```

Modified:

```
def qualify_trends(portfolio_id: str, as_of_date: str):
    # Load portfolio-specific configuration
    portfolio_config = load_portfolio_config(portfolio_id)
    filters = portfolio_config['universe_filters']

    # Apply portfolio-specific filters
    qualified = apply_filters(
        asset_classes=filters['asset_classes'],
        exchanges=filters['exchanges'],
        min_market_cap=filters['min_market_cap'],
        max_beta=filters['max_beta'],
        # ... etc
    )

    # Save with portfolio ID
    save_qualified_symbols(qualified, portfolio_id, as_of_date)
```

Script 7: Momentum Ranker

Status: Minor modification required

Modified:

```
def rank_momentum(portfolio_id: str, as_of_date: str):
    # Load qualified symbols for this portfolio
    qualified = load_qualified_symbols(portfolio_id, as_of_date)

    # Load strategy parameters
    config = load_portfolio_config(portfolio_id)
    params = config['strategy_parameters']

    # Rank using portfolio-specific SMA parameters
    ranked = calculate_momentum(
        symbols=qualified,
        sma_slow=params['sma_slow'],
        as_of_date=as_of_date
    )

    # Save with portfolio ID
    save_momentum_ranked(ranked, portfolio_id, as_of_date)
```

Script 8: Position Sizer

Status: Moderate modification required

Modified:

```
def calculate_position_sizes(portfolio_id: str, as_of_date: str):
    # Load portfolio state
    portfolio_state = load_portfolio_state(portfolio_id)
    account_equity = portfolio_state['total_value']

    # Load portfolio configuration
    config = load_portfolio_config(portfolio_id)
    params = config['strategy_parameters']
    limits = config['risk_limits']

    # Load momentum ranked symbols
    ranked = load_momentum_ranked(portfolio_id, as_of_date)

    # Calculate position count
    position_count = min(
        params['position_count_max'],
        max(params['position_count_min'], int(account_equity / 5000))
    )

    # Select top N
    top_n = ranked[:position_count]

    # Calculate sizes with portfolio-specific parameters
    position_sizes = []
    for symbol in top_n:
        size = calculate_volatility_adjusted_size(
            symbol=symbol,
            account_equity=account_equity,
            risk_per_position=params['risk_per_position'],
```

```

        max_position_size=limits['max_position_size'],
        min_position_size=limits['min_position_size']
    )
    position_sizes.append(size)

# Save with portfolio ID
save_position_sizes(position_sizes, portfolio_id, as_of_date)

```

Script 9: Stop-Loss Calculator

Status: Minor modification required

Modified:

```

def calculate_stops(portfolio_id: str, as_of_date: str):
    # Load portfolio configuration
    config = load_portfolio_config(portfolio_id)
    params = config['strategy_parameters']

    # Load current positions
    positions = load_portfolio_positions(portfolio_id)

    # Calculate stops with portfolio-specific multipliers
    for position in positions:
        # Initial stop
        if not position['has_initial_stop']:
            initial_stop = calculate_initial_stop(
                entry_price=position['entry_price'],
                atr=position['atr'],
                multiplier=params['initial_stop_multiplier']
            )
            position['initial_stop'] = initial_stop

        # Trailing stop
        profit_pct = (position['current_price'] - position['entry_price']) / position['entry_price']
        if profit_pct >= params['trailing_stop_activation']:
            trailing_stop = calculate_trailing_stop(
                current_price=position['current_price'],
                atr=position['atr'],
                multiplier=params['trailing_stop_multiplier'],
                current_stop=position['current_stop']
            )
            position['current_stop'] = max(position['current_stop'], trailing_stop)
            position['trailing_stop_active'] = True

    # Save updated positions
    save_portfolio_positions(positions, portfolio_id, as_of_date)

```

Script 11: Monthly Rebalancer

Status: Major modification required

Current: Single portfolio rebalancing

Modified: Multi-portfolio batch rebalancing

New Structure:

```

def monthly_rebalancing(as_of_date: str, portfolio_ids: list = None):
    """
    Run rebalancing for multiple portfolios

    Args:
        as_of_date: Rebalancing date
        portfolio_ids: List of portfolio IDs, or None for all active
    """
    # Load all portfolio configurations
    all_configs = load_all_portfolio_configs()

    # Filter to active portfolios
    if portfolio_ids is None:
        portfolio_ids = [p for p, config in all_configs.items() if config['active']]

    logger.info(f"Rebalancing {len(portfolio_ids)} portfolios: {portfolio_ids}")

    # Process each portfolio
    rebalancing_results = {}

```

```

for portfolio_id in portfolio_ids:
    logger.info(f"Processing portfolio: {portfolio_id}")

    try:
        # Load portfolio configuration
        config = all_configs[portfolio_id]

        # Check rebalancing schedule
        if not should_rebalance(portfolio_id, as_of_date, config):
            logger.info(f"{portfolio_id}: Not scheduled for rebalancing today")
            continue

        # Check circuit breakers (portfolio-specific)
        breakers_triggered = check_circuit_breakers(portfolio_id, config['circuit_breakers'])
        if breakers_triggered:
            logger.warning(f"{portfolio_id}: Circuit breakers triggered: {breakers_triggered}")
            rebalancing_results[portfolio_id] = {
                'status': 'HALTED',
                'reason': f"Circuit breakers: {breakers_triggered}"
            }
            continue

        # Run qualification (portfolio-specific filters)
        qualified = qualify_trends(portfolio_id, as_of_date)

        # Rank by momentum
        ranked = rank_momentum(portfolio_id, as_of_date)

        # Calculate position sizes
        position_sizes = calculate_position_sizes(portfolio_id, as_of_date)

        # Calculate stops
        calculate_stops(portfolio_id, as_of_date)

        # Generate exit signals
        exits = generate_exit_signals(portfolio_id, as_of_date)

        # Create rebalancing recommendations
        recommendations = generate_recommendations(
            portfolio_id=portfolio_id,
            as_of_date=as_of_date,
            exits=exits,
            new_positions=position_sizes,
            config=config
        )

        rebalancing_results[portfolio_id] = {
            'status': 'SUCCESS',
            'exits': len(exits),
            'entries': len([p for p in position_sizes if p['action'] == 'BUY']),
            'recommendations': recommendations
        }

    except Exception as e:
        logger.error(f"{portfolio_id}: Rebalancing failed: {e}")
        rebalancing_results[portfolio_id] = {
            'status': 'ERROR',
            'error': str(e)
        }

# Check for cross-portfolio issues
check_aggregate_risk(portfolio_ids, as_of_date)

return rebalancing_results

```

Script 12: Recommendation Report Generator

Status: Major modification required

New Features:

- Generate individual portfolio reports
- Generate consolidated report showing all portfolios
- Cross-portfolio analysis section

Modified:

```

def generate_recommendation_reports(as_of_date: str, rebalancing_results: dict):
    """
    Generate reports for all portfolios + consolidated view

```

```

"""
reports_generated = []

# Generate individual portfolio reports
for portfolio_id, results in rebalancing_results.items():
    if results['status'] == 'SUCCESS':
        report_path = generate_portfolio_report(
            portfolio_id=portfolio_id,
            as_of_date=as_of_date,
            recommendations=results['recommendations']
        )
        reports_generated.append(report_path)

# Generate consolidated report
consolidated_path = generate_consolidated_report(
    as_of_date=as_of_date,
    all_results=rebalancing_results
)
reports_generated.append(consolidated_path)

return reports_generated

```

Script 13: Execution Logger

Status: Moderate modification required

Modified:

```

def log_execution(portfolio_id: str, trade: dict):
    """
    Log trade execution for specific portfolio
    """
    # Load portfolio state
    portfolio_state = load_portfolio_state(portfolio_id)

    # Update position or create new
    if trade['action'] == 'BUY':
        add_position(portfolio_state, trade)
    elif trade['action'] == 'SELL':
        remove_position(portfolio_state, trade)

    # Update cash
    portfolio_state['cash'] += trade['proceeds'] if trade['action'] == 'SELL' else -trade['cost']

    # Recalculate totals
    portfolio_state['equity_value'] = sum(p['position_value'] for p in portfolio_state['positions'])
    portfolio_state['total_value'] = portfolio_state['cash'] + portfolio_state['equity_value']

    # Save updated state
    save_portfolio_state(portfolio_state, portfolio_id)

    # Log trade to portfolio-specific execution log
    log_trade(portfolio_id, trade)

    # Update aggregate state
    update_aggregate_state()

```

Script 14: Daily Risk Monitor

Status: Major modification required

New: Monitor each portfolio individually + aggregate monitoring

Modified:

```

def daily_monitoring():
    """
    Monitor all active portfolios + aggregate risk
    """
    # Load all portfolio configurations
    all_configs = load_all_portfolio_configs()
    active_portfolios = [p for p, c in all_configs.items() if c['active']]

    logger.info(f"Monitoring {len(active_portfolios)} portfolios")

    all_alerts = []

```

```

# Monitor each portfolio individually
for portfolio_id in active_portfolios:
    portfolio_alerts = monitor_portfolio(portfolio_id, all_configs[portfolio_id])
    if portfolio_alerts:
        all_alerts.extend(portfolio_alerts)

# Monitor aggregate risks
aggregate_alerts = monitor_aggregate_risk(active_portfolios)
if aggregate_alerts:
    all_alerts.extend(aggregate_alerts)

# Send alerts if any triggered
if all_alerts:
    send_alerts(all_alerts)

# Generate daily monitoring report
generate_daily_report(active_portfolios, all_alerts)

return all_alerts

def monitor_portfolio(portfolio_id: str, config: dict) -> List:
    """
    Monitor individual portfolio health
    """
    alerts = []

    # Load portfolio state
    state = load_portfolio_state(portfolio_id)
    latest_data = load_latest_market_data()

    # Check 1: Stop-loss violations (portfolio-specific)
    for position in state['positions']:
        current_price = latest_data[position['symbol']]['close']
        if current_price <= position['current_stop']:
            alerts.append({
                'portfolio_id': portfolio_id,
                'severity': 'HIGH',
                'type': 'stop_loss_hit',
                'symbol': position['symbol'],
                'current_price': current_price,
                'stop_price': position['current_stop'],
                'action': 'Exit at next market open'
            })

    # Check 2: Portfolio drawdown (portfolio-specific threshold)
    peak = state['metadata'].get('peak_equity', state['initial_capital'])
    current = state['total_value']
    drawdown = (current - peak) / peak

    if drawdown < config['circuit_breakers']['portfolio_drawdown']:
        alerts.append({
            'portfolio_id': portfolio_id,
            'severity': 'CRITICAL',
            'type': 'circuit_breaker_drawdown',
            'drawdown': drawdown,
            'threshold': config['circuit_breakers']['portfolio_drawdown'],
            'action': 'Halt new entries for 5 days'
        })

    # Check 3: Concentration risk (portfolio-specific threshold)
    sorted_positions = sorted(state['positions'], key=lambda x: x['position_value'], reverse=True)
    top_3_value = sum(p['position_value'] for p in sorted_positions[:3])
    concentration = top_3_value / state['equity_value']

    if concentration > config['circuit_breakers']['concentration_creep']:
        alerts.append({
            'portfolio_id': portfolio_id,
            'severity': 'MEDIUM',
            'type': 'concentration_risk',
            'concentration': concentration,
            'threshold': config['circuit_breakers']['concentration_creep'],
            'action': 'Review position sizing'
        })

    # Additional checks...

    return alerts

```

Purpose: Monitor cross-portfolio risks

New Script:

```
def monitor_aggregate_risk(portfolio_ids: list) -> list:
    """
    Monitor risks that span multiple portfolios
    """
    alerts = []

    # Load aggregate state
    aggregate = load_aggregate_state()

    # Check 1: Cross-portfolio correlation
    if aggregate['cross_portfolio_analysis']['correlation_matrix']['CONSERVATIVE_AGGRESSIVE'] > 0.85:
        alerts.append({
            'portfolio_id': 'AGGREGATE',
            'severity': 'HIGH',
            'type': 'high_cross_correlation',
            'correlation': aggregate['cross_portfolio_analysis']['correlation_matrix']['CONSERVATIVE_AGGRESSIVE'],
            'action': 'Portfolios moving together - diversification benefit lost'
        })

    # Check 2: Overlapping positions (concentrated risk)
    overlapping = aggregate['cross_portfolio_analysis']['overlapping_symbols']
    if len(overlapping) > 10:
        # Calculate total exposure to overlapping positions
        total_exposure = 0
        for portfolio_id in portfolio_ids:
            state = load_portfolio_state(portfolio_id)
            for position in state['positions']:
                if position['symbol'] in overlapping:
                    total_exposure += position['position_value']

        exposure_pct = total_exposure / aggregate['aggregate']['total_value']

        if exposure_pct > 0.40:
            alerts.append({
                'portfolio_id': 'AGGREGATE',
                'severity': 'MEDIUM',
                'type': 'overlapping_positions_concentration',
                'overlapping_count': len(overlapping),
                'total_exposure_pct': exposure_pct,
                'symbols': overlapping,
                'action': 'High overlap reduces diversification'
            })

    # Check 3: Total account drawdown
    combined_dd = aggregate['aggregate']['combined_max_drawdown']
    if combined_dd < -0.25:
        alerts.append({
            'portfolio_id': 'AGGREGATE',
            'severity': 'CRITICAL',
            'type': 'aggregate_drawdown',
            'drawdown': combined_dd,
            'action': 'All portfolios experiencing significant drawdown'
        })

    # Check 4: Cash depletion across all portfolios
    total_cash_pct = aggregate['aggregate']['total_cash'] / aggregate['aggregate']['total_value']
    if total_cash_pct < 0.02:
        alerts.append({
            'portfolio_id': 'AGGREGATE',
            'severity': 'MEDIUM',
            'type': 'low_aggregate_cash',
            'cash_pct': total_cash_pct,
            'action': 'Limited dry powder for opportunities'
        })

    return alerts
```

Scripts 16-21: Backtesting & Validation

Status: Minor modification required

Change: Run backtests per portfolio configuration

Modified:

```
def backtest_portfolio(portfolio_id: str, start_date: str, end_date: str):
    """
    Backtest specific portfolio configuration
    """
    # Load portfolio configuration
    config = load_portfolio_config(portfolio_id)

    # Run backtest with portfolio-specific parameters
    results = run_backtest(
        start_date=start_date,
        end_date=end_date,
        strategy_params=config['strategy_parameters'],
        universe_filters=config['universe_filters'],
        risk_limits=config['risk_limits'],
        initial_capital=config['initial_capital']
    )

    # Save results with portfolio ID
    save_backtest_results(results, portfolio_id)

    return results
```

Scripts 22-24: Performance Management

Status: Major modification required

Change: Calculate performance per portfolio + aggregate

Script 22: Performance Attribution

```
def performance_attribution(portfolio_id: str = None):
    """
    Calculate attribution for specific portfolio or all
    """
    if portfolio_id:
        # Single portfolio attribution
        return calculate_portfolio_attribution(portfolio_id)
    else:
        # Aggregate attribution
        all_configs = load_all_portfolio_configs()
        active = [p for p, c in all_configs.items() if c['active']]

        # Calculate for each
        individual_attributions = {}
        for pid in active:
            individual_attributions[pid] = calculate_portfolio_attribution(pid)

        # Calculate aggregate
        aggregate_attribution = calculate_aggregate_attribution(individual_attributions)

        return {
            'individual': individual_attributions,
            'aggregate': aggregate_attribution
        }
```

Script 23: Risk Analytics

```
def risk_analytics(portfolio_id: str = None):
    """
    Calculate risk metrics for specific portfolio or aggregate
    """
    if portfolio_id:
        # Single portfolio risk
        return calculate_portfolio_risk(portfolio_id)
    else:
        # Aggregate risk including cross-portfolio correlation
        return calculate_aggregate_risk()
```

Script 24: Performance Dashboard

```
def generate_dashboard(portfolio_id: str = None):
    """
    Generate dashboard for specific portfolio or consolidated view
    """
    if portfolio_id:
        # Single portfolio dashboard
        return generate_portfolio_dashboard(portfolio_id)
    else:
```

```
# Consolidated dashboard with all portfolios
return generate_consolidated_dashboard()
```

AGGREGATE MONITORING

New Script 25: Cross-Portfolio Analytics

Purpose: Analyze relationships and risks across multiple portfolios

Key Metrics

1. Cross-Portfolio Correlation

Correlation = Correlation(Portfolio_A_Daily_Returns, Portfolio_B_Daily_Returns)

Target: <0.70 (diversification benefit)

Warning: >0.85 (portfolios too similar)

2. Diversification Benefit

Expected_Combined_Vol = sqrt(w_A^2 * Vol_A^2 + w_B^2 * Vol_B^2 + 2*w_A*w_B*Vol_A*Vol_B*Corr)

Actual_Combined_Vol = StdDev(Combined_Returns)

Diversification_Benefit = (Expected - Actual) / Expected

Target: >10% (meaningful diversification)

3. Overlapping Positions Analysis

Overlap_Count = Number of symbols held in multiple portfolios

Overlap_Exposure = Sum of all overlapping position values

Warning if:

- Overlap_Count > 10 symbols
- Overlap_Exposure > 40% of total equity

4. Aggregate Drawdown

Combined_Peak = max(Total_Value_All_Portfolios)

Combined_Current = sum(Current_Value_Each_Portfolio)

Aggregate_Drawdown = (Combined_Current - Combined_Peak) / Combined_Peak

Alert if: <-25%

5. Portfolio Drift Analysis

Actual_Allocation = Current_Value_Portfolio_i / Total_Value_All

Target_Allocation = Initial_Capital_Portfolio_i / Total_Initial_Capital

Drift = Actual - Target

Rebalance if drift > 10% for any portfolio

Aggregate Monitoring Dashboard

Consolidated View Components

Portfolio Comparison Table:

| Portfolio | Value | Return YTD | Sharpe | Max DD | Beta | Positions |

|-----|-----|-----|-----|-----|-----|-----|

| Conservative | €77.7K | 12.5% | 1.15 | -8% | 0.85 | 18 |

| Aggressive | €68.4K | 18.5% | 1.32 | -15% | 1.25 | 12 |

| **Aggregate** | **€146.1K** | **15.3%** | **1.23** | **-11%** | **1.03** | **30** |

Cross-Portfolio Correlation Matrix:

Conservative Conservative Aggressive

Conservative 1.00 0.62

Aggressive 0.62 1.00

Overlapping Positions:

- 5 symbols held in both portfolios
- Total overlap exposure: 28% of equity
- Top overlaps: AAPL (€9.8K), MSFT (€7.2K), GOOGL (€5.5K)

Asset Class Allocation (Aggregate):

- Stocks: 65% (target: 60-70%)
 - ETFs: 28% (target: 25-35%)
 - Crypto: 7% (target: 5-10%)
 - Cash: 13% (target: 5-15%)
-

REPORTING STRUCTURE

Individual Portfolio Reports

Report: `reports/rebalancing/CONSERVATIVE/2026-02_recommendations.pdf`

Contents (unchanged structure, portfolio-specific data):

1. Portfolio Summary
 - Portfolio ID, name, description
 - Current value, return, Sharpe, drawdown
 - Benchmark comparison (portfolio-specific)
 2. Recommended Exits
 3. Recommended Entries
 4. Positions to Hold
 5. Risk Metrics (portfolio-specific limits)
 6. Approval Section
-

Consolidated Report

Report: `reports/rebalancing/CONSOLIDATED/2026-02_all_portfolios.pdf`

New Contents:

Section 1: Executive Summary

- All portfolios overview table
- Aggregate metrics
- Cross-portfolio correlation
- Overlapping positions

Section 2: Portfolio-by-Portfolio Details

- Conservative Portfolio (2 pages summary)
- Aggressive Portfolio (2 pages summary)
- Additional portfolios...

Section 3: Cross-Portfolio Analysis

- Correlation matrix
- Overlapping positions detail
- Aggregate allocation pie chart
- Combined performance vs individual
- Diversification benefit calculation

Section 4: Aggregate Risk Assessment

- Total exposure by asset class
- Total exposure by sector
- Aggregate max drawdown
- Cross-portfolio circuit breaker status
- Cash position across all portfolios

Section 5: Recommendations Summary

- Total trades across all portfolios
- Execution schedule (which portfolio when)
- Aggregate impact on cash
- Total transaction costs

Section 6: Approval

- Sign-off required for EACH portfolio
- Aggregate risk manager sign-off
- Consolidated approval record

Performance Dashboards

Individual Dashboard

reports/performance/CONSERVATIVE/dashboard.html

Same components as before:

- Equity curve
- Monthly returns heatmap
- Drawdown chart
- Rolling Sharpe
- Position P&L table

Consolidated Dashboard

reports/performance/CONSOLIDATED/dashboard.html

New Components:

Tab 1: Aggregate Performance

- Combined equity curve (all portfolios stacked)
- Aggregate monthly returns
- Combined drawdown
- Weighted Sharpe ratio over time

Tab 2: Portfolio Comparison

- Side-by-side equity curves
- Comparison table (returns, Sharpe, DD)
- Correlation heatmap
- Scatter plot (return vs risk)

Tab 3: Cross-Portfolio Analysis

- Overlapping positions breakdown
- Sector exposure across all portfolios
- Asset class allocation (aggregate)
- Contribution to total return by portfolio

Tab 4: Individual Portfolios

- Dropdown selector
- Full individual dashboard for selected portfolio

OPERATIONAL WORKFLOW

Daily Workflow (Modified)

6:00 AM - Data Update

```
# Unchanged - single download serves all
python scripts/01_download_eodhd_bulk.py --mode incremental
```

6:05 AM - Risk Monitoring (Modified)

```
# Monitor all active portfolios + aggregate
python scripts/14_daily_monitoring.py --all-portfolios

# Or specific portfolio
python scripts/14_daily_monitoring.py --portfolio CONSERVATIVE
```

6:15 AM - Review Alerts

```
Manual:
  Check email/SMS for alerts
  Review daily monitoring report
  Take action on stop-loss hits (per portfolio)
```

Monthly Workflow (Modified)

Last Trading Day of Month - Rebalancing

5:00 AM - Pre-Rebalancing

```
# Ensure data current
python scripts/01_download_eodhd_bulk.py --mode incremental

# Update fundamentals if new symbols detected
python scripts/02_download_yahoo_fundamentals.py --mode incremental
```

9:00 AM - Generate Recommendations

```
# Option A: All portfolios at once
python scripts/11_monthly_rebalancing.py --all-portfolios --date 2026-02-28

# Option B: Specific portfolios
python scripts/11_monthly_rebalancing.py --portfolios CONSERVATIVE,AGGRESSIVE --date 2026-02-28

# Option C: One at a time
python scripts/11_monthly_rebalancing.py --portfolio CONSERVATIVE --date 2026-02-28
python scripts/11_monthly_rebalancing.py --portfolio AGGRESSIVE --date 2026-02-28
```

10:00 AM - Human Review

Manual:

1. Review individual portfolio reports

- CONSERVATIVE: Check exits, entries, sizing
- AGGRESSIVE: Check exits, entries, sizing

2. Review consolidated report

- Cross-portfolio correlation
- Overlapping positions
- Aggregate risk metrics
- Total cash required for entries

3. Make approval decision

- APPROVE all portfolios
- APPROVE some, REJECT others
- MODIFY recommendations
- REJECT all

4. Sign approval documents (per portfolio)

First Trading Day of New Month - Execution

9:30 AM - Execute Exits (Prioritized)

```
# Execute exits for all portfolios (market orders)
python scripts/13_log_execution.py --portfolio CONSERVATIVE --execute-exits
python scripts/13_log_execution.py --portfolio AGGRESSIVE --execute-exits
```

3:30 PM - Execute Entries (Limit Orders)

```
# Place entry orders for all portfolios
python scripts/13_log_execution.py --portfolio CONSERVATIVE --execute-entries
python scripts/13_log_execution.py --portfolio AGGRESSIVE --execute-entries
```

Quarterly Workflow (Modified)

Performance Review

```
# Generate performance reports for all portfolios
python scripts/22_performance_attribution.py --all-portfolios --period 2026-Q1
python scripts/23_risk_analytics.py --all-portfolios --period 2026-Q1
python scripts/24_performance_dashboard.py --all-portfolios --period 2026-Q1

# Generate consolidated view
python scripts/24_performance_dashboard.py --consolidated --period 2026-Q1
```

Cross-Portfolio Analysis

```
# New: Cross-portfolio analytics
python scripts/25_cross_portfolio_analytics.py --period 2026-Q1
```

IMPLEMENTATION ROADMAP

Phase 1: Configuration Framework (Week 1)

Tasks:

1. Create config/portfolios.json structure
2. Define schemas for portfolio configuration
3. Create portfolio loading utilities
4. Test configuration validation

Deliverables:

- portfolios.json with 2 portfolio definitions
- Configuration loader functions
- Validation tests

Phase 2: Data Architecture Changes (Week 2)

Tasks:

1. Restructure data/ directory (portfolios subdirs)
2. Update portfolio_state.json schema
3. Create aggregate_state.json schema
4. Implement portfolio-specific data saving

Deliverables:

- New directory structure created
- Updated schemas documented
- Migration script for existing data

Phase 3: Core Script Modifications (Weeks 3-4)

Tasks:

1. Modify Scripts 6-10 (add portfolio_id parameter)
2. Update Script 11 (multi-portfolio rebalancing)
3. Update Script 12 (individual + consolidated reports)
4. Update Script 13 (portfolio-tagged execution logging)
5. Update Script 14 (multi-portfolio monitoring)

Deliverables:

- All core scripts support portfolio_id
- Backward compatibility maintained
- Unit tests for multi-portfolio logic

Phase 4: Aggregate Monitoring (Week 5)

Tasks:

1. Implement Script 25 (Cross-Portfolio Analytics)
2. Create aggregate state calculation
3. Build cross-portfolio correlation monitoring
4. Implement overlapping position analysis

Deliverables:

- Script 25 complete and tested
 - Aggregate monitoring dashboard
 - Alert system for cross-portfolio risks
-

Phase 5: Reporting Infrastructure (Week 6)

Tasks:

1. Update Scripts 22-24 for multi-portfolio
2. Create consolidated dashboard
3. Build portfolio comparison views
4. Implement cross-portfolio analysis reports

Deliverables:

- Individual portfolio reports (unchanged)
 - New consolidated report template
 - Interactive consolidated dashboard
-

Phase 6: Testing & Validation (Week 7)

Tasks:

1. End-to-end testing with 2 portfolios
2. Test circuit breakers per portfolio
3. Test aggregate monitoring
4. Validate performance calculations
5. Test rebalancing with different schedules

Deliverables:

- Comprehensive test suite
 - Validation report
 - Bug fixes
-

Phase 7: Production Deployment (Week 8)

Tasks:

1. Deploy to production environment
2. Initialize 2 portfolios with real capital allocation
3. Run first parallel rebalancing
4. Monitor for 1 month
5. Iterate based on learnings

Deliverables:

- Production system managing 2 portfolios
 - Operational procedures documented
 - Post-launch report
-

SUMMARY

Changes Required

Minimal Changes:

- ✓ Scripts 1-3: No changes (data download/consolidation shared)
- ✓ Script 5: No changes (indicators shared)
- ✓ Scripts 16-21: Minor changes (add portfolio_id parameter)
- ✓ Script 15: Minor changes (add portfolio filter)

Moderate Changes:

- ● Scripts 6-10: Add portfolio_id, load portfolio-specific config
- ● Script 13: Tag executions with portfolio_id

Major Changes:

- ⚠ **Script 4: Portfolio-specific universe screening (11 filter categories)**
- ⚠ Script 11: Batch multi-portfolio rebalancing
- ⚠ Script 12: Individual + consolidated reports
- ⚠ Script 14: Multi-portfolio + aggregate monitoring
- ⚠ Scripts 22-24: Per-portfolio + aggregate performance

New Components:

- 🌟 config/portfolios.json (with universe_filters section)
 - 🌟 Script 25: Cross-Portfolio Analytics
 - 🌟 Aggregate state tracking
 - 🌟 Consolidated reporting
 - 🌟 Universe overlap monitoring
 - 🌟 Screening reports per portfolio
-

Benefits of Multi-Portfolio Architecture

1. Diversification

- Different strategies reduce correlation
- Conservative + Aggressive = smoother combined returns

2. Testing

- Run parameter variants in production
- Real-world A/B testing
- Learn which approaches work best

3. Risk Segmentation

- Conservative portfolio: Preserve capital
- Aggressive portfolio: Seek growth
- Allocate capital based on risk tolerance

4. Flexibility

- Easy to add new portfolios (just add config)
- Can pause/activate portfolios individually
- Each portfolio independently managed

5. Scalability

- Same architecture supports 2 or 20 portfolios
 - No fundamental changes needed to add more
 - Linear scaling of complexity
-

Effort Estimate

Total Implementation Time: 8 weeks

Breakdown:

- Week 1: Configuration framework
- Week 2: Data architecture
- Weeks 3-4: Core script modifications
- Week 5: Aggregate monitoring
- Week 6: Reporting
- Week 7: Testing
- Week 8: Production deployment

Team Size: 1 developer (can be parallelized with 2)

Complexity: Medium (well-defined changes, clear patterns)

END OF DOCUMENT

Version: Multi-Portfolio Extension v1.0

Date: February 2026

Status: Complete Architecture for 2+ Portfolios

Next Steps: Begin Phase 1 implementation or approve architecture